L06 · 词嵌入 Embedding

> *"把文字变成数字的魔法"*

📋 本节目标

学完本节，你将能够：

1. 理解从 One-Hot 到 Dense Embedding 的演进过程
2. 掌握 `nn.Embedding` 的本质——查表操作
3. 理解嵌入维度 d_model=768 的含义
4. 掌握权重共享（Weight Tying）的原理和好处
5. 计算 Embedding 层的参数量

🔗 前置知识

- [L05 · Tokenizer 分词器]——知道 token id 是什么
- 线性代数基础：向量、矩阵、矩阵乘法
- PyTorch 基础：`nn.Module`、`nn.Linear`

---

1. 从文字到向量：为什么需要 Embedding?

1.1 问题：模型不认识整数

上一节我们用 Tokenizer 把文字变成了整数序列：

```
"今天天气真好" → [356, 1892, 478, 23]
```

但这些整数对模型来说**毫无意义**。数字 356 并不比 357 更接近"今"的含义。我们需要一种方式，让每个 token 变成一个**有意义的向量**，使得语义相似的词在向量空间中距离更近。

1.2 一个类比

想象你是一个图书管理员。每本书都有一个编号（ISBN），但编号本身不告诉你这本书讲什么。你需要一个系统，给每本书贴上多个维度的标签：

- 科幻程度：0.8
- 浪漫程度：0.2
- 知识密度：0.9
- 阅读难度：0.7
- ...（768 个维度）

这样，《三体》和《银河帝国》的标签就会很相似，而《三体》和《红楼梦》的标签就很不同。

Embedding 就是给每个 token 贴上一组"多维标签"的过程。

---

2. One-Hot：最朴素的方案

2.1 什么是 One-Hot 编码?

最简单的想法：用一个长度为 vocab_size 的向量来表示每个 token，对应位置为 1，其余全为 0。

```
词表：["猫", "狗", "鸟", "鱼"]  (vocab_size=4)

"猫" → [1, 0, 0, 0]
"狗" → [0, 1, 0, 0]
"鸟" → [0, 0, 1, 0]
"鱼" → [0, 0, 0, 1]
```

2.2 One-Hot 的致命缺陷

1. **维度灾难**：MiniMind 的 vocab_size=6400，每个 token 就是一个 6400 维的向量。Qwen2 的 vocab_size=151,643 就更夸张了。

2. **无法表达语义**：任意两个 One-Hot 向量的内积都是 0（正交），"猫"和"狗"的距离与"猫"和"数学"的距离完全一样。

cos_sim("猫", "狗") = cos_sim("猫", "数学") = 0

1. **极度稀疏**：6400 维的向量里只有 1 个位置非零，99.98% 的数据是 0，浪费计算和存储。

---

3. Dense Embedding：稠密向量的魔法

3.1 核心思想

与其用稀疏的 One-Hot，不如用一个**较短但稠密**的向量来表示每个 token：

```
词表大小: 6400
嵌入维度: 768

"猫" → [0.23, -0.45, 0.12, ..., 0.67]  (768维)
"狗" → [0.21, -0.41, 0.15, ..., 0.63]  (768维)  ← 和"猫"很接近!
"数学"→ [-0.55, 0.32, 0.89, ..., -0.11] (768维) ← 和"猫"很远
```

3.2 数学描述

本质上，Embedding 就是一个矩阵乘法：

$\mathbf{e}$ = one_hot(x) $\cdot \mathbf{W}_E$

其中： - x 是 token id（整数） - one_hot(x) 是 $1 \times V$ 的 One-Hot 向量（V = vocab_size） - $\mathbf{W}_E$ 是 $V \times d$ 的嵌入矩阵（d = d_model） - $\mathbf{e}$ 是 $1 \times d$ 的嵌入向量

但因为 One-Hot 向量只有一个位置是 1，所以这个矩阵乘法**等价于从矩阵中取出第 x 行**——这就是"查表"操作。

3.3 查表 vs 矩阵乘法

```
import torch
import torch.nn as nn

V, d = 6400, 768
W = nn.Embedding(V, d)

# 方法1：查表（实际实现）
token_id = 356
embedding = W(torch.tensor(token_id))  # 直接取第356行

# 方法2：矩阵乘法（数学等价，但更慢）
one_hot = torch.zeros(V)
one_hot[356] = 1.0
embedding_v2 = one_hot @ W.weight  # 矩阵乘法

# embedding ≈ embedding_v2（结果相同）
```

实际实现中，`nn.Embedding` 用的是查表，不是矩阵乘法——因为查表快得多！

---

4. nn.Embedding 详解

4.1 API 说明

```
nn.Embedding(num_embeddings, embedding_dim)
```

- `num_embeddings`：词表大小（有多少行，即多少个 token）
- `embedding_dim`：嵌入维度（每行有多少列）

4.2 嵌入矩阵的形状

对于 MiniMind：

```
nn.Embedding(6400, 768)

嵌入矩阵 W_E 的形状: (6400, 768)
  → 6400 行：每个 token 一行
  → 768 列：每个 token 用 768 维向量表示
```

可以这样理解：这个矩阵是一本"字典"，有 6400 个词条，每个词条用 768 个数字描述。

4.3 输入输出示意

```
# 输入：一批 token id
input_ids = torch.tensor([[356, 1892, 478, 23]])  # shape: (1, 4)

# 通过 Embedding 层
embed = nn.Embedding(6400, 768)
output = embed(input_ids)  # shape: (1, 4, 768)

# 每个 token id 变成了一个 768 维向量
```

```
输入 shape:  (batch_size, seq_len)           = (1, 4)
输出 shape:  (batch_size, seq_len, d_model)  = (1, 4, 768)
```

4.4 参数量计算

Embedding 参数量 = vocab_size × d_model = $6400 \times 768 = 4,915,200 \approx 5M$

对于 MiniMind 的 64M 总参数来说，Embedding 占了约 **7.7%**。如果用 Qwen2 的词表（151,643），参数量将是：

$151,643 \times 768 = 116,461,824 \approx 116M$

这已经**超过 MiniMind 的总参数量**了！这就是为什么小模型必须用小词表。

---

5. 嵌入维度 d_model 的含义

5.1 d_model=768 代表什么?

d_model 是模型的"宽度"，贯穿整个 Transformer 的每一层。它决定了模型能表达多少信息。

一个直觉性的理解： - d_model=2：只能区分"正面/负面"这种粗粒度语义 - d_model=768：能捕捉非常细粒度的语义差异 - d_model=4096（如 LLaMA）：语义空间更大，表达能力更强

5.2 为什么是 768?

768 = 12 × 64 = 8 × 96。选择这个数字是因为： 1. 能被多头注意力的头数整除（MiniMind 用 8 个头，768/8=96） 2. 对于 64M 参数的模型来说是一个合理的宽度 3. 与 BERT-base（也是 768）相同，是一个经过充分验证的值

5.3 常见模型的 d_model

模型	d_model	参数量
BERT-base	768	110M
GPT-2	768	117M
MiniMind	**768**	**64M**
LLaMA-7B	4,096	7B
Qwen2-72B	8,192	72B

---

6. 权重共享（Weight Tying）

6.1 什么是权重共享?

在 Transformer 语言模型中，有两个地方用到了词表维度的矩阵：

1. **输入端 Embedding 层**（`embed_tokens`）：token id → 向量
2. 形状：(vocab_size, d_model) = (6400, 768)
3. **输出端 LM Head 层**（`lm_head`）：向量 → token 概率
4. 形状：(d_model, vocab_size) = (768, 6400)

权重共享就是让这两个矩阵共用同一组参数。

6.2 为什么可以共享?

从语义上理解:

- Embedding 层:给定一个 token id,找到它的"语义表示"
- LM Head 层:给定一个"语义表示",找到最匹配的 token

这两个操作是**互逆的**!Embedding 是"查表"(id → 向量),LM Head 是"反查"(向量 → id 的概率)。用同一张表做正查和反查,完全合理。

6.3 权重共享的好处

1. **减少参数量**:省掉了一半的词嵌入参数
2. 不共享:5M + 5M = 10M 参数
3. 共享后:仅 5M 参数
4. 对 MiniMind(64M)来说节省了约 **7.7%** 的参数
5. **语义一致性**:输入和输出使用同一个语义空间,训练更稳定
6. **正则化效果**:减少了自由参数,有轻微的正则化作用,防止过拟合

6.4 MiniMind 中的实现

在 `model/model_minimind.py` 中:

```
class MiniMindModel(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.embed_tokens = nn.Embedding(config.vocab_size, config.dim)
        # ... 其他层 ...
        self.lm_head = nn.Linear(config.dim, config.vocab_size, bias=False)

        # 权重共享!
        self.lm_head.weight = self.embed_tokens.weight
```

最后一行是关键:`lm_head.weight` 直接指向 `embed_tokens.weight`,两者在内存中是同一个张量。训练时梯度会同时从两条路径回传到这个共享参数。

---

7. Word2Vec:历史的起点(简介)

7.1 为什么提 Word2Vec?

在 Transformer 出现之前（2013 年），Google 的 Word2Vec 是词嵌入的里程碑。虽然现在 LLM 中不直接用 Word2Vec，但它的核心思想"用上下文定义词义"至今仍是基础。

7.2 核心思想

> **"告诉我你的朋友是谁，我就知道你是谁"**

Word2Vec 通过预测上下文来学习词向量： - **CBOW**：用周围的词预测中间的词 - **Skip-gram**：用中间的词预测周围的词

7.3 著名的"向量算术"

Word2Vec 发现了令人惊叹的性质：

$\vec{\text{King}} - \vec{\text{Man}} + \vec{\text{Woman}} \approx \vec{\text{Queen}}$

这说明嵌入空间捕捉到了"性别"这个语义维度。类似的：

$\vec{\text{北京}} - \vec{\text{中国}} + \vec{\text{日本}} \approx \vec{\text{东京}}$

7.4 与现代 Embedding 的区别

特性	Word2Vec	现代 LLM Embedding
训练方式	独立训练	与模型联合训练
上下文感知	静态（同一个词只有一个向量）	动态（经过 Transformer 后随上下文变化）
用途	特征提取	模型的第一层
词表	词级别	子词（BPE）级别

---

8. MiniMind 源码解读

8.1 模型配置中的关键参数

在 `model/model_minimind.py` 的 `LMConfig` 类中：

```
class LMConfig:
    dim: int = 768           # d_model，嵌入维度
    vocab_size: int = 6400   # 词表大小
```

8.2 Embedding 层的定义

```
class MiniMindModel(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.embed_tokens = nn.Embedding(config.vocab_size, config.dim)
```

这一行创建了一个 (6400, 768) 的嵌入矩阵。

8.3 Forward 中的 Embedding

```
def forward(self, input_ids, ...):
    h = self.embed_tokens(input_ids)  # (batch, seq_len) → (batch, seq_len, 768)
    # h 就是每个 token 的嵌入向量，接下来送入 Transformer 层
```

8.4 权重共享的实现

```
class MiniMindModel(nn.Module):
    def __init__(self, config):
        # ...
        self.lm_head = nn.Linear(config.dim, config.vocab_size, bias=False)
        self.lm_head.weight = self.embed_tokens.weight  # 权重共享
```

8.5 数据流小结

```
token ids     Embedding      Transformer       LM Head          概率
[356,1892] → [768d,768d] → [768d,768d] → [6400d,6400d] → softmax → 预测
            查表            核心处理         用共享权重投影
```

🎤 面试考点

Q1：`nn.Embedding` 的本质是什么？

参考答案：`nn.Embedding` 本质上是一个查表（lookup table）操作。它维护一个 (vocab_size, embedding_dim) 的权重矩阵，给定 token id 就返回该行的向量。在数学上等价于 One-Hot 向

量与嵌入矩阵的乘法，但实现上用的是索引查找，效率远高于矩阵乘法。

Q2：MiniMind 的 Embedding 参数量是多少？占总参数量的比例？

参考答案：Embedding 参数量 = 6400 × 768 = 4,915,200 ≈ 5M。如果不做权重共享，Embedding + lm_head = 10M，占 64M 总参数的约 15.6%。做了权重共享后只有 5M，占约 7.7%。

Q3：什么是权重共享（Weight Tying）？为什么要这么做？

参考答案：权重共享是指 Embedding 层和 LM Head 层使用同一个参数矩阵。好处有三：（1）节省参数量，对小模型尤其重要；（2）保持输入输出语义空间的一致性；（3）有轻微的正则化效果。这是因为 Embedding（id→向量）和 LM Head（向量→概率）是互逆操作，使用同一组参数在语义上是合理的。

Q4：为什么 One-Hot 编码在大规模 NLP 中不可行？

参考答案：One-Hot 有三个致命缺陷：（1）维度等于词表大小，大词表下维度爆炸；（2）任意两个向量都正交，无法表达语义相似性；（3）极度稀疏，浪费存储和计算。Dense Embedding 用低维稠密向量解决了这些问题。

Q5：嵌入维度 d_model 的选择有什么讲究？

参考答案：d_model 决定了模型的"宽度"和表达能力。选择时需要考虑：（1）必须能被注意力头数整除（MiniMind：768/8=96）；（2）要与模型总参数量匹配——太大浪费参数，太小表达力不足；（3）通常是 2 的幂或其倍数，便于 GPU 并行计算。

Q6：Word2Vec 和 Transformer 中的 Embedding 有什么区别？

参考答案：Word2Vec 的 Embedding 是静态的，同一个词无论在什么上下文中向量都一样；Transformer 的 Embedding 只是第一层，经过多层 Self-Attention 后，同一个词在不同上下文中会有不同的表示（上下文相关）。此外，Word2Vec 是独立训练的，而 Transformer 的 Embedding 是与整个模型端到端联合训练的。

✅ 自测题

1. **填空**：`nn.Embedding(6400, 768)` 内部维护的权重矩阵形状是 _____。
2. **计算**：如果不做权重共享，MiniMind 的 Embedding + lm_head 总共有多少参数？
3. **判断**：`nn.Embedding` 的本质是矩阵乘法。（对/错？）

4. **简答**：权重共享为什么不会导致 Embedding 和 lm_head 的功能冲突？
5. **思考**：如果 d_model 从 768 增大到 1024，模型的哪些部分会受影响？

▶ 查看答案

🔮 下一节预告

token 的嵌入向量已经有了语义信息，但 Transformer 的训练是否稳定呢？深度网络中梯度很容易爆炸或消失。下一节 **L07 · RMSNorm 归一化**，我们将学习 MiniMind 如何用一个简洁的归一化层来守护训练的稳定性。

⬅️ L05 · Tokenizer 分词器 | 目录 | L07 · RMSNorm 归一化 ➡️